RTL Design & Verification of MESI Cache Coherence Protocol with Next-Line Prefetching

MID-SEMESTER PROJECT EVALUATION REPORT

Course: Computer Architecture [COURSE CODE PLACEHOLDER]

Group Number: [PLACEHOLDER]

Group Members: Aman Bhandari (MT26151), Shubham Kumar (MT26184), Gaurav Yadav (MT26162)

Instructor: [PLACEHOLDER]

Mentor / TA: [PLACEHOLDER, if applicable]

Date: 8 October 2026

1. Proposed System Architecture

The proposed system models a two-core cache hierarchy. Each core/request generator has a private L1 cache implementing MESI coherence. The two private L1 caches communicate through a shared snooping bus. The shared bus also serializes competing lower-level requests before forwarding them to one shared L2 cache. The L2 cache connects to main memory.

A separate L2 arbiter module is not part of the final architecture. Arbitration is implemented as functionality inside the shared bus/interconnect.

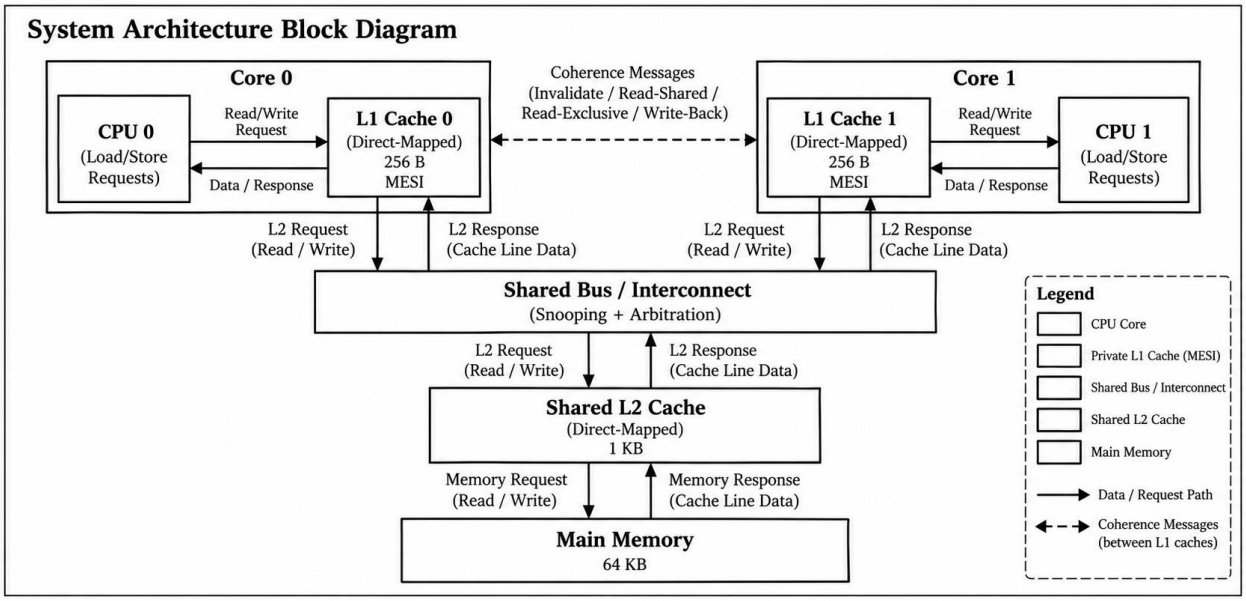


Figure 1. Proposed two-core MESI cache hierarchy with shared snooping bus/interconnect.

1.1 Major RTL Blocks

RTL Block	Role
cache_system.v	Top-level integration of two L1 caches, shared bus, L2 cache, and main memory.
l1_cache.v	Private L1 cache. Stores data, tags, and MESI state; handles hits, misses, snoops, invalidation, intervention, and dirty eviction.
shared_bus.v	Shared request path and snooping/interconnect logic. Handles simultaneous L1 requests through internal arbitration.
l2_cache.v	Shared direct-mapped L2 cache between the private L1 caches and main memory.
main_memory.v	Main-memory model with configured access latency.
cache_defs.vh	Common architectural parameters and constants.

2. Architectural Design Parameters

Parameter	Selected Value
Number of processor/request-generator interfaces	2
Address width	16 bits
Processor data width	32 bits
Private L1 cache size	256 B per core
L1 associativity	Direct-mapped
Number of L1 lines	16
Shared L2 cache size	1 KB
L2 associativity	Direct-mapped
Number of L2 lines	64
Main-memory size	64 KB
Cache-line size	16 B
Words per cache line	4 x 32-bit words
Replacement policy	Implicit/deterministic direct-mapped replacement
Write policy	Write-back
Write-miss policy	Write-allocate
L1 access latency	1 cycle
L2 access latency	5 cycles
Main-memory access latency	20 cycles
Coherence protocol	MESI
Interconnect	Shared snooping bus with internal arbitration
Next-line prefetcher	Planned for final-stage implementation; not part of current mid-sem RTL

2.1 Address Breakdown

The 16-bit address width is a design choice for the RTL prototype. Since main memory is 64 KB and byte-addressable, 16 address bits cover the complete 64 KB address space.

Level	Tag	Index	Word Offset	Byte Offset
L1	address[15:8] (8 bits)	address[7:4] (4 bits)	address[3:2] (2 bits)	address[1:0] (2 bits)
L2	address[15:10] (6 bits)	address[9:4] (6 bits)	address[3:2] (2 bits)	address[1:0] (2 bits)

3. Justification of Architectural Parameters

3.1 L1 Cache: 256 B, Direct-Mapped

A small 256 B private L1 keeps the RTL, cache arrays, address decoding, and verification manageable while still allowing the design to demonstrate hits, misses, conflicting addresses, MESI state changes, and dirty replacement. With a 16 B line, each L1 contains 16 lines. Direct mapping gives every memory block exactly one possible L1 location, so replacement is deterministic and no LRU/FIFO metadata is required.

3.2 Shared L2 Cache: 1 KB, Direct-Mapped

The 1 KB L2 is larger than either private L1 and provides 64 cache lines with the selected 16 B line size. This preserves a clear L1-to-L2 hierarchy while keeping simulation and debugging practical. A direct-mapped L2 also keeps the mid-sem RTL focused on hierarchy and coherence behavior rather than replacement bookkeeping.

3.3 Main Memory: 64 KB

A 16-bit byte address spans 2^16 = 65,536 byte locations, so the main-memory model is 64 KB. This lets the full 16-bit address space be represented directly.

3.4 Cache-Line Size: 16 B

A 16 B cache line contains four 32-bit processor words. This is large enough to demonstrate line-based fills, write-backs, and coherence while remaining small enough that complete line transfers are easy to inspect in simulation.

3.5 Replacement Policy

A direct-mapped organization is used for both L1 and L2. In this organization, each memory block maps to one fixed cache index, so replacement is deterministic. This is useful for directed verification of the coherence controller because conflicting addresses can be selected deliberately to force a known replacement. For example, addresses 0x0040 and 0x0140 map to the same L1 index in the current design; this property is used in the dirty-eviction test to verify that a line in the Modified state is written back before it is replaced. Since there is only one candidate line at each index, a victim-selection algorithm such as LRU is not applicable.

3.6 Write-Back and Write-Allocate

Write-back is a natural match for MESI because a line in the Modified state may contain a newer value than the lower-level cache or memory. Modified data is written back when required, including intervention and dirty eviction. Write-allocate ensures that a write miss first obtains the cache line before the processor's write is applied.

3.7 Latency Selection

The selected latencies are L1 = 1 cycle, L2 = 5 cycles, and main memory = 20 cycles. They preserve the expected ordering L1 < L2 < memory and make the difference between a hit at each hierarchy level visible in simulation without making tests unnecessarily long.

4. MESI State-Transition / FSM

Each private L1 cache line maintains one of four MESI coherence states.

State	Meaning in This Design
I - Invalid	The L1 does not hold a valid copy of the line.
S - Shared	The line is clean and may also be present in the other L1.
E - Exclusive	The line is clean and is held only by this L1.
M - Modified	This L1 holds the newest modified copy; the lower level may be stale.

RTL state encoding: I = 00, S = 01, E = 10, M = 11.

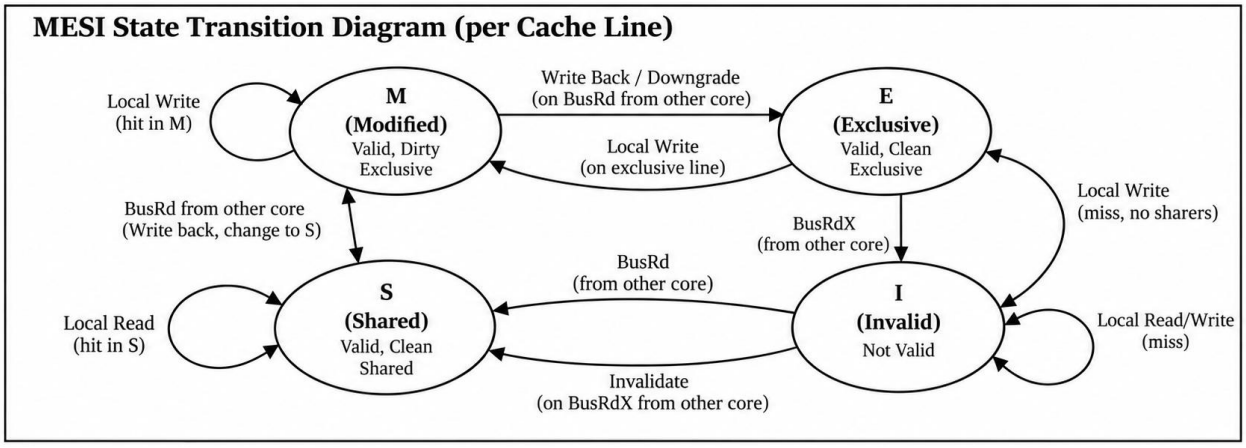


Figure 2. MESI state-transition diagram for a private L1 cache line.

4.1 Important Transitions Covered by Current Verification

Starting State / Situation	Event	Result / Action
I	Local processor read; no peer copy	Line is fetched and enters E.
I	Local processor read; peer has a copy	Requester receives line in S; peer remains/becomes S.
E	Local processor write	E -> M.
E	Other core reads same line	E -> S; requester receives S.
S	Local processor write	S -> M and the other cached copy is invalidated.
M	Other core reads same line	Modified data is intervened/written back; owner M -> S and requester receives S.
M	Conflicting line replaces it	Dirty line is written back before replacement.

5. Test Plan

Eight dedicated Verilog testbenches are used for the mid-semester functional demonstration. Each demonstration has its own concise terminal result and its own VCD waveform, keeping the report evidence isolated and reproducible.

Testbench / Scenario	Stimulus	What It Verifies	Expected Result	Status
cold_miss_tb.v	Cold read of 0x0040	L1/L2 miss path and main-memory access	L1 miss; L2 miss; memory accessed; correct data returned	PASS
l1_hit_tb.v	Fill 0x0040, then read 0x0040 again	Private L1 hit	L1 hit; no L2 or memory access	PASS
l2_hit_tb.v	Read 0x0080, read conflicting 0x0180, then read 0x0080	Dedicated shared-L2 hit	L1 miss; L2 hit; no memory access	PASS
two_l1_tb.v	Both cores issue reads in the same cycle	Shared-bus request serialization	Both requests complete with correct data	PASS
mesi_read_tb.v	Core 0 reads, then Core 1 reads the same line	MESI read sharing	E,I -> S,S	PASS
mesi_write_tb.v	Establish S,S, then Core 0 writes 0xDEADBEEF	MESI ownership and invalidation	S,S -> M,I	PASS
mesi_intervention_tb.v	Core 0 modifies line; Core 1 later reads it	Modified-line intervention	Core 1 receives 0xDEADBEEF; final S,S	PASS
dirty_eviction_tb.v	Modify 0x0040; read conflicting 0x0140	Dirty write-back before replacement	0x0040 written back; 0x0140 installed	PASS

5.1 Directed Scenarios to Demonstrate

1. Cold read: L1 miss -> L2 miss -> main-memory access -> cache-line refill.
2. Repeated read after refill: L1 hit with no L2 or main-memory access.
3. L2 hit: 0x0080 is removed from L1 by conflicting address 0x0180 while remaining resident in L2; re-reading 0x0080 produces an L1 miss, L2 hit, and no main-memory access.
4. Two simultaneous requests from the two private L1 caches complete correctly through the shared bus.
5. MESI read sharing: first private copy enters Exclusive; a peer read changes the two copies to Shared.
6. MESI write and invalidation: a write to a Shared line changes the writer to Modified and invalidates the peer copy.
7. Modified-line intervention: a peer read receives the newest modified value and both copies finish in Shared.
8. Dirty eviction: a conflicting line forces a Modified victim to be written back before direct-mapped replacement.

6. Project Schedule and Team Roles

Milestone	Work Item	Status / Target
M1	Select hierarchy and architectural parameters	Completed
M2	Implement main-memory model	Completed
M3	Implement shared L2 cache	Completed
M4	Implement private L1 cache and hierarchy	Completed
M5	Integrate two L1s with shared bus	Completed
M6	Implement MESI read/write/snoop behavior	Completed
M7	Implement intervention and dirty eviction/write-back	Completed
M8	Finalize dedicated demonstration testbenches and one-waveform-per-demonstration structure	Completed
M9	Capture final terminal outputs, waveform screenshots, diagrams, references, and report evidence	In progress
M10	Implement next-line prefetcher	Final-stage work
M11	Compare baseline vs prefetch-enabled performance/traffic/latency	Final-stage work
M12	Final simulation checks, presentation, and viva preparation	Final-stage work

6.1 Team Allocation

Team Member	Primary Responsibility	Secondary / Verification Responsibility
Aman Bhandari - MT26151	Private L1 cache and MESI controller	MESI read sharing, write/invalidation, and intervention verification
Shubham Kumar - MT26184	Shared L2 cache and main-memory subsystem	Hierarchy, L2-hit, and dirty-writeback verification
Gaurav Yadav - MT26162	Shared bus/interconnect and system integration	Simultaneous-request verification and integration checks

7. RTL Implementation

The current mid-sem RTL implements the L1, L2, and main-memory subsystem in Verilog, together with the shared snooping bus and top-level integration. Functional evidence is organized as one dedicated demonstration testbench and one waveform per report scenario.

File	Implemented Functionality	Mid-Sem Evidence
cache_defs.vh	Common widths, cache sizes, line size, and latency parameters	Parameter values used by all RTL and demonstration testbenches
main_memory.v	64 KB memory model with read/write interface and 20-cycle latency	Cold-miss demonstration
l2_cache.v	1 KB direct-mapped shared L2 with line storage and 5-cycle latency	Cold-miss and dedicated L2-hit demonstrations
l1_cache.v	256 B private L1, MESI state storage, hit/miss handling, write-back/write-allocate, snoops, intervention, dirty eviction	L1-hit, MESI, intervention, and dirty-eviction demonstrations
shared_bus.v	Shared L2 access path, snooping/coherence signaling, simultaneous-request arbitration	Simultaneous-request and MESI demonstrations
cache_system.v	Integrates two L1s, bus, L2, and memory	All two-core coherence and integration demonstrations

7.1 Current Implementation Scope

- Two private L1 caches are integrated and independently service processor/request-generator operations.
- Shared L2 and main memory provide the lower levels of the hierarchy.
- Basic read and write operations are implemented.
- MESI read sharing, write invalidation, modified intervention, and dirty eviction are implemented.
- The next-line prefetcher is intentionally deferred to the final-stage deliverable.

8. Functional Demonstration

Functional verification is presented using eight dedicated testbenches. Each testbench generates a separate VCD waveform. Where a demonstration requires cache state to be established first, the terminal output also shows the short setup sequence before the final target access, making the result self-contained and easy to reproduce.

8.1 Cache Miss -> L2 -> Main Memory -> Refill

The dedicated cold-miss demonstration reads address 0x0040 from an initially empty hierarchy. The request misses in the private L1 and shared L2, accesses main memory, returns the 16-byte line through L2, and refills the requesting L1. The configured component latencies are L1 = 1 cycle, L2 = 5 cycles, and main memory = 20 cycles; the testbench also reports the full end-to-end transaction latency.

Dedicated demonstration files

Testbench: tb/cold_miss_tb.v

Run command: ./scripts/run_cold_miss.sh

Waveform: waves/cold_miss.vcd

Evidence: 0x0040 produces L1 miss -> L2 miss -> main-memory access -> line refill; waveform shows request/address, L1 and L2 activity, memory request/response, returned data, and completion.

The terminal result reports the requested address, returned value, L1/L2 miss status, main-memory access, and measured end-to-end transaction latency. The end-to-end value includes request, transfer, refill, and handshake overhead and is not the configured 20-cycle memory latency itself.

8.2 L1 Hit

The L1-hit demonstration first fills address 0x0040 and then reads the same address again. The target access is satisfied by the private L1 with no L2 or main-memory request. The configured L1 latency is 1 cycle; the terminal-reported transaction latency includes the surrounding request/ready handshake.

Dedicated demonstration files

Testbench: tb/l1_hit_tb.v

Run command: ./scripts/run_l1_hit.sh

Waveform: waves/l1_hit.vcd

Evidence: after a setup fill of 0x0040, the target read is an L1 hit; no L2 or main-memory request occurs during the target access.

8.3 L2 Access / L2 Hit

The L2-hit demonstration first reads 0x0080, then reads 0x0180. These addresses map to the same direct-mapped L1 index but different L2 indices, so 0x0180 replaces 0x0080 only in L1. Re-reading 0x0080 therefore produces an L1 miss and an L2 hit without a main-memory access. The configured L2 latency is 5 cycles; the testbench separately reports end-to-end transaction latency.

Dedicated demonstration files

Testbench: tb/l2_hit_tb.v

Run command: ./scripts/run_l2_hit.sh

Waveform: waves/l2_hit.vcd

Evidence: 0x0080 is loaded, 0x0180 replaces it in L1 only, and the final read of 0x0080 produces L1 miss -> L2 hit with no main-memory access.

8.4 Simultaneous Two-Core Requests

The simultaneous-request demonstration issues a read from each core in the same cycle. The shared bus serializes the lower-level requests without losing either transaction, and both cores receive the expected data values.

Verified example: Core 0 address 0x0040 -> 0xAAAAAAAA; Core 1 address 0x0080 -> 0xBBBBBBBB.

Dedicated demonstration files

Testbench: tb/two_l1_tb.v

Run command: ./scripts/run_two_l1.sh

Waveform: waves/two_l1.vcd

Evidence: Core 0 reads 0x0040 -> 0xAAAAAAAA while Core 1 simultaneously reads 0x0080 -> 0xBBBBBBBB; the shared bus serializes the lower-level requests and both complete correctly.

8.5 MESI Read Sharing

The MESI read-sharing demonstration first lets Core 0 read an uncached line, producing E,I. Core 1 then reads the same line, causing Core 0 to transition E -> S while Core 1 transitions I -> S. Both caches finish in Shared with the same data.

Dedicated demonstration files

Testbench: tb/mesi_read_tb.v

Run command: ./scripts/run_mesi_read.sh

Waveform: waves/mesi_read.vcd

Evidence: first read produces E,I; the peer read changes the states to S,S. MESI encoding is I=00, S=01, E=10, M=11.

8.6 MESI Write and Invalidation

The MESI write/invalidation demonstration first establishes S,S for address 0x0040. Core 0 then writes 0xDEADBEEF, obtains ownership in Modified, and invalidates Core 1. The final line states are M,I.

Dedicated demonstration files

Testbench: tb/mesi_write_tb.v

Run command: ./scripts/run_mesi_write.sh

Waveform: waves/mesi_write.vcd

Evidence: after S,S is established, Core 0 writes 0xDEADBEEF; Core 0 becomes M and the peer line becomes I.

8.7 Modified-Line Intervention

The modified-line intervention demonstration places 0xDEADBEEF in Core 0's L1 in Modified while the L2 copy remains 0x11111111. When Core 1 reads the line, Core 0 intervenes and writes back the modified data before Core 1 completes. Core 1 receives 0xDEADBEEF and both L1 copies finish in Shared.

Dedicated demonstration files

Testbench: tb/mesi_intervention_tb.v

Run command: ./scripts/run_mesi_intervention.sh

Waveform: waves/mesi_intervention.vcd

Evidence: Core 0 holds 0xDEADBEEF in M while L2 is stale at 0x11111111; Core 1 reads and receives 0xDEADBEEF, L2 is updated, and both caches finish in S.

8.8 Dirty Eviction / Write-Back

The dirty-eviction demonstration modifies address 0x0040 to 0xDEADBEEF and then reads conflicting address 0x0140, which maps to the same direct-mapped L1 index. Before 0x0140 replaces the victim, the Modified 0x0040 line is written back to L2. The final checks confirm that L2 contains 0xDEADBEEF and that 0x0140 is installed at the conflicting L1 index.

Dedicated demonstration files

Testbench: tb/dirty_eviction_tb.v

Run command: ./scripts/run_dirty_eviction.sh

Waveform: waves/dirty_eviction.vcd

Evidence: 0x0040 is Modified with 0xDEADBEEF; conflicting read 0x0140 forces write-back of 0x0040 before 0x0140 replaces it at the same L1 index.

9. References and Resources

The references below document the resources consulted during design, implementation, and verification. [Replace the remaining placeholders with the exact sources used by the team.]

Type	Resource	How It Was Used
Textbook / paper	[PLACEHOLDER: MESI/cache-coherence reference]	MESI states, snooping events, and coherence behavior
Research paper	[PLACEHOLDER: paper title, authors, venue/year]	Background on cache coherence / multicore systems
Video / lecture	[PLACEHOLDER: NPTEL or other lecture actually used]	Conceptual understanding of snooping and MESI transitions
Documentation	Icarus Verilog documentation [exact URL/reference to be inserted]	Compilation and Verilog simulation
Documentation	Surfer documentation [exact URL/reference to be inserted]	VCD waveform inspection
Repository / code reference	[PLACEHOLDER: only if an external repository was actually consulted]	[PLACEHOLDER: specific idea learned]
Other	[PLACEHOLDER]	[PLACEHOLDER]

Reference format: [PLACEHOLDER - apply the citation style specified for the course, or use one consistent academic style throughout.]